

## Syllabus

**Introduction to Software Engineering:** Introduction: FAQ's about software engineering, IEEE / ACM code of software engineering ethics, Process activities;

**Requirements gathering and analysis:** Software Requirements Specification (SRS), Functional and Non-Functional requirements, User requirements, System requirements, Interface specification, Characteristics and components of SRS, Structure of SRS (IEEE format).

**Case study:** Develop SRS (IEEE format) for any real-world application.

## Introduction

### Software :

Software is:

- (1) instructions (computer programs) that when executed provide desired features, function, and performance;
- (2) data structures that enable the programs to adequately manipulate information, and
- (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

## Characteristics of Software

### 1. *Software is developed or engineered; it is not manufactured in the classical sense.*

Although some similarities exist between software development and hardware manufacturing, the two activities are fundamentally different. In both activities, high quality is achieved through good design, but the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software. Both activities are dependent on people, but the relationship between people applied and work accomplished is entirely different. Both activities require the construction of a "product," but the approaches are different. Software costs are concentrated in engineering. This means that software projects cannot be managed as if they were manufacturing projects.

### 2. *Software doesn't "wear out."*

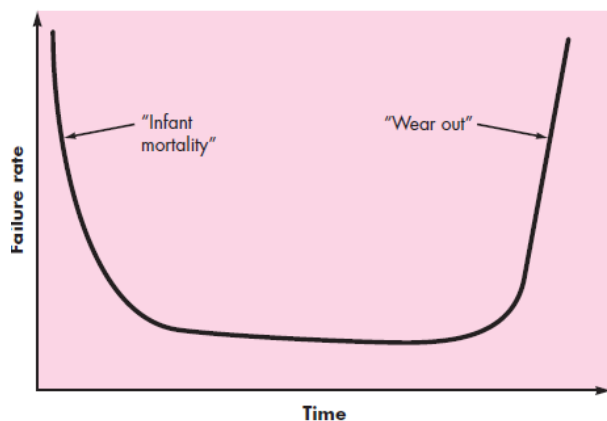
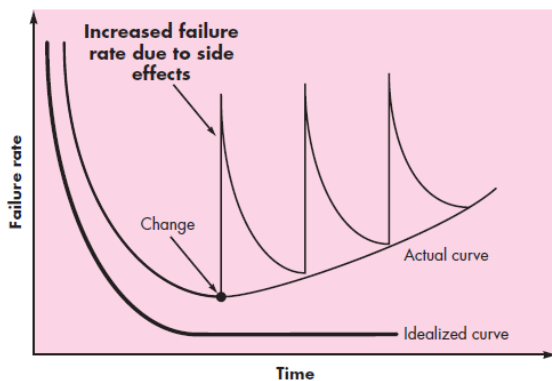


Figure 1.1 depicts failure rate as a function of time for hardware. The relationship, often called the "bathtub curve," indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects); defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the hardware begins to *wear out*. Software is not susceptible to the environmental maladies that cause hardware to wear out. In theory, therefore,

the failure rate curve for software should take the form of the “idealized curve” shown in Figure 1.2.



Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected and the curve flattens as shown. The idealized curve is a gross oversimplification of actual failure models for software. However, the implication is clear—software doesn’t wear out. But it does *deteriorate*!

During its life, software will undergo change. As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike as shown in the “actual curve” (Figure 1.2). Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise—the software is deteriorating due to change. Another aspect of wear illustrates the difference between hardware and software. When a hardware component wears out, it is replaced by a spare part. There are no software spare parts. Every software failure indicates an error in design or in the process through which design was translated into machine executable code. Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance.

### **3. Although the industry is moving toward component-based construction, most software continues to be custom built.**

As an engineering discipline evolves, a collection of standard design components is created. Standard screws and off-the-shelf integrated circuits are only two of thousands of standard components that are used by mechanical and electrical engineers as they design new systems. The reusable components have been created so that the engineer can concentrate on the truly innovative elements of a design, that is, the parts of the design that represent something new. In the hardware world, component reuse is a natural part of the engineering process. In the software world, it is something that has only begun to be achieved on a broad scale. A software component should be designed and implemented so that it can be reused in many different programs. Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new applications from reusable parts .

For example, today’s interactive user interfaces are built with reusable components that enable the creation of graphics windows, pull-down menus, and a wide variety of interaction mechanisms. The data structures and processing detail required to build the interface are contained within a library of reusable components for interface construction.

## **The Importance of Software**

- Software packs and delivers *Information*
- Software is the differentiator
  - Hardware is general-purpose, software is application-specific

- Hardware is becoming a commodity
- Software is often costlier than the hardware.
- Software is crucial for business success

### **The Software Crisis**

- Early software development was 'personalized'
- As sophistication of applications grew,
  - programs grew and complexity
  - Maintenance became difficult
  - Taking more time
  - Costing more than initially estimated.
  - Software delivered only to fail.
  - Errors found were difficult to fix

### **Need For Engineering Approach**

- Programming was seen as an art.
- Design happened mentally, not on paper.
- Business growth demanded better software development.
- Programming techniques failed for large projects.
- Issues were people, estimation, and management.
- Engineering inspired "Software Engineering" discipline.

## **Software Application domains**

### **System Software**

Provides a platform for other software to run and manages hardware resources (e.g., operating systems, device drivers, compilers).

### **2. Business / Enterprise Applications**

Supports organizational processes such as accounting, payroll, inventory, CRM, and ERP systems.

### **3. Scientific and Engineering Applications**

Used for numerical computation, simulation, modeling, and analysis in science and engineering domains.

### **4. Embedded Systems**

Software embedded within hardware devices to control and monitor their operations (e.g., washing machines, cars, medical devices).

### **5. Real-Time Systems**

Responds to inputs within strict time constraints; correctness depends on both logical results and timing (e.g., air traffic control).

### **6. Web Applications**

Runs on web browsers and servers, providing services over the internet (e.g., e-commerce, social media, online banking).

### **7. Mobile Applications**

Designed for smartphones and tablets, utilizing mobile OS features such as GPS, camera, and touch interfaces.

### **8. Artificial Intelligence Applications**

Implements intelligent behavior such as learning, reasoning, and decision-making (e.g., chatbots, recommender systems).

### **9. Data-Intensive / Big Data Applications**

Processes, analyzes, and manages large volumes of structured and unstructured data for insights and predictions.

### **10. Gaming and Multimedia Applications**

Focuses on graphics, audio, video, animation, and interactive user experiences (e.g., games, AR/VR systems).

## Software Engineering

Software engineering is the application of a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software *i.e., the application of engineering to software*.

The purpose of software engineering(1/2)

- Software helps customers achieve business goals.
  - Example 1: Hospital manager needs patient records.
  - Example 2: Factory manager needs production coordination.
- Engineers must understand customer business needs
- This task requires
  - Learn new disciplines and processes quickly.
  - Communicate, abstract problems, and propose solutions.
  - Design systems that adapt over time.

## Relevance of software engineering

- The economies of ALL developed nations are dependent on software.
- More and more systems are software controlled
- Software engineering is concerned with theories, methods and tools for professional software development.
- Expenditure on software represents a significant fraction of GNP in all developed countries

## Software costs

- Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- Software engineering is concerned with cost-effective software development

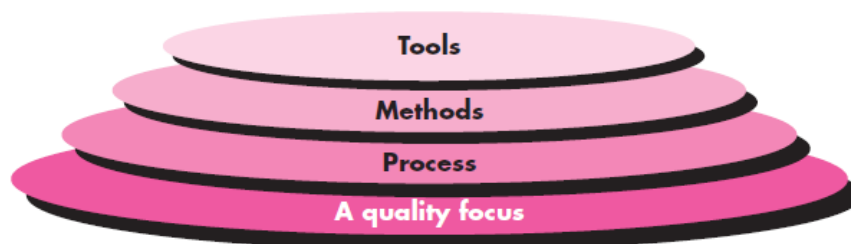


Fig 1.3

## Quality Focus (Bedrock Layer)

Software engineering rests on a strong organizational commitment to quality, supported by practices like TQM and Six Sigma. Quality philosophies promote continuous improvement, leading to more effective and mature software engineering practices

## Process Layer (Foundation)

The software process forms the foundation and acts as the “glue” that integrates all other layers. Process provides a framework for project management, defining milestones, work products, quality assurance, and change control.

## Methods Layer

Software engineering methods provide the technical “how-to” for activities such as requirements analysis, design, coding, testing, and maintenance.

## Tools Layer

Tools offer automated or semi-automated support for processes and methods; integrated tools form CASE (Computer-Aided Software Engineering) systems.

## Software project failure

- *Increasing system complexity*  
As new software engineering techniques help us to build larger, more complex systems, the demands change. Systems have to be built and delivered more quickly; larger, even more complex systems are required; systems have to have new capabilities that were previously thought to be impossible.
- *Failure to use software engineering methods*  
It is fairly easy to write computer programs without using software engineering methods and techniques. Many companies have drifted into software development as their products and services have evolved. They do not use software engineering methods in their everyday work. Consequently, their software is often more expensive and less reliable than it should be.

## Frequently asked questions about software engineering

| Question                                                                    | Answer                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is software?                                                           | Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.                                                  |
| What are the attributes of good software?                                   | Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.                                                                  |
| What is software engineering?                                               | Software engineering is an engineering discipline that is concerned with all aspects of software production.                                                                                            |
| What are the fundamental software engineering activities?                   | Software specification, software development, software validation and software evolution.                                                                                                               |
| What is the difference between software engineering and computer science?   | Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.                                            |
| What is the difference between software engineering and system engineering? | System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process. |

| Question                                                       | Answer                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What are the key challenges facing software engineering?       | Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.                                                                                                                                                                                                                                                                                              |
| What are the costs of software engineering?                    | Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.                                                                                                                                                                                                                                                       |
| What are the best software engineering techniques and methods? | While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another. |
| What differences has the web made to software engineering?     | The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.                                                                                                                                                          |

### Software products

- ✧ Generic products
  - Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
  - Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.
- ✧ Customized products
  - Software that is commissioned by a specific customer to meet their own needs.
  - Examples – embedded control systems, air traffic control software, traffic monitoring systems.

### Product specification

- ✧ Generic products
  - The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.
- ✧ Customized products
  - The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

## Essential attributes of good software

| Product characteristic     | Description                                                                                                                                                                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Maintainability            | Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.                                              |
| Dependability and security | Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system. |
| Efficiency                 | Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.                                                                                     |
| Acceptability              | Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.                                                                                              |

## Software process activities

- Software specification, where customers and engineers define the software that is to be produced and the constraints on its operation.
- Software development, where the software is designed and programmed.
- Software validation, where the software is checked to ensure that it is what the customer requires.
- Software evolution, where the software is modified to reflect changing customer and market requirements

## General issues that affect software

- ✧ Heterogeneity
  - Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.
- ✧ Business and social change
  - Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.
- ✧ Security and trust
  - As software is intertwined with all aspects of our lives, it is essential that we can trust that software.
- ✧ Scale
  - Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.



### **Software engineering ethics**

- ✧ Software engineering involves wider responsibilities than simply the application of technical skills.
- ✧ Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- ✧ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

### **Issues of professional responsibility**

- ✧ Confidentiality
  - Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- ✧ Competence
  - Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.
- ✧ Intellectual property rights
  - Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.
- ✧ Computer misuse
  - Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

### **ACM/IEEE Code of Ethics**

- ✧ The professional societies in the US have cooperated to produce a code of ethical practice.
- ✧ Members of these organisations sign up to the code of practice when they join.
- ✧ The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

### **Rationale for the code of ethics**

- *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

## **Software Engineering Code of Ethics and Professional Practice**

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

### **PREAMBLE**

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

### Ethical Principles

1. PUBLIC - Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.